

Deep Learning — Week 2 Notes

Sigmoid Neurons · Loss Functions · Feedforward Networks · Gradient Descent

NPTEL / IIT · CS7015-style course · Exam-focused revision sheet

1 · The Sigmoid Neuron

From hard threshold to smooth output

A perceptron gives a **hard 0/1 decision**. A **sigmoid neuron** replaces the step with a smooth curve, so the output is a **probability-like score** in the open interval (0, 1) instead of a rigid class label.

$$z = w_1x_1 + w_2x_2 + b \quad \hat{p} = \sigma(z) = 1 / (1 + e^{-z})$$

- **Maps any real number** → (0, 1): this is exactly why sigmoid suits a probability score.
- Key anchor: $\sigma(0) = 0.5$. Sigmoid is **monotonically increasing**.
- Positive z → output above 0.5; negative z → output below 0.5.

Threshold trick: the rule " $\hat{p} \geq 0.5$ " is **exactly equivalent to** $z \geq 0$. You never need to compute the exponential — just check the sign of z .

Effect of the bias b

The bias shifts the whole decision line.

- **More negative bias** → lowers z for the same inputs
→ **fewer records** cross the threshold.
- More positive bias → more activations.

Worked check (z)

Model: $z = 3x_1 + 2x_2 - 2$.

For $x = (0.4, 0.6)$: $z = 1.2 + 1.2 - 2 = \mathbf{0.4} \rightarrow \hat{p} > 0.5 \rightarrow$ flagged.

For $x = (0.1, 0.4)$: $z = 0.3 + 0.8 - 2 = \mathbf{-0.9} \rightarrow$ **not** flagged.

2 · Loss Functions & Squared Error

Measuring how wrong a prediction is

For real-valued prediction (regression), we use **squared error**. It penalises large mistakes heavily because the error is squared.

$$L = (y - \hat{y})^2 \quad \text{MSE} = (1/n) \sum_i (y_i - \hat{y}_i)^2$$

$$\text{Gradient w.r.t. prediction: } \partial L / \partial \hat{y} = 2(\hat{y} - y)$$

Worked example — validation orders

Order	y	$\hat{y}$	error (y- $\hat{y}$)	squared
1	12	10	+2	4
2	20	24	-4	16
3	15	13	+2	4
4	18	21	-3	9

Sum of squared errors = $4 + 16 + 4 + 9 = 33 \rightarrow \text{MSE} = 33 / 4 = 8.25$. Order 2 is the largest single contributor (16).

Gradient check: for $y = 20$, $\hat{y} = 24 \rightarrow \partial L / \partial \hat{y} = 2(24 - 20) = +8$. A positive gradient means the prediction is too high and should be pushed down.

Comparing models: lower MSE = better fit. A model predicting (11, 21, 14, 19) has errors all of size 1, so $\text{MSE} = 1.00$ — much better than 8.25.

3 · Feedforward Neural Networks

Hidden layers and order of computation

A feedforward network transforms inputs through a **hidden layer** before producing the output. Computation flows in one direction: **inputs** $\rightarrow$ **hidden units** $\rightarrow$ **output**.

$$\begin{aligned}h_1 &= \sigma(x_1 + x_2 - 1) & h_2 &= \sigma(2x_1 - x_2) \\ \hat{y} &= \sigma(3h_1 + 2h_2 - 2)\end{aligned}$$

- **ORDER** Hidden units h_1, h_2 are computed **first**, then the output.
- **WHY NONLINEAR** Each hidden unit applies a **nonlinear sigmoid** to a weighted sum — this lets the network represent nonlinear transformations a single perceptron cannot.
- **RANGE** Final σ keeps $\hat{y}$ bounded in **(0, 1)**, not an unrestricted value.

Trace by hand ($x = 1, 1$): input to $h_1 = 1 + 1 - 1 = 1$.

($x = 1, 0$): input to $h_2 = 2(1) - 0 = 2$. Only the pre-activation is asked — apply σ only if the output value is needed.

Exam trap: if hidden units used *no* nonlinearity (identity), stacking layers would collapse into a single linear map. The nonlinearity is what makes depth useful.

4 · Gradient Descent

Learning by following the slope

Gradient descent updates each parameter in the direction that **reduces the loss**. The gradient tells us the local rate of change of L with respect to the parameter.

$$w \leftarrow w - \eta \cdot (dL/dw)$$

where η is the **learning rate** (step size).

Worked one-parameter example

Model $\hat{y} = wx$, with $x = 3$, $y = 15$, so loss $L(w) = (15 - 3w)^2$.

Quantity	Computation	Result
L at $w = 4$	$\hat{y} = 12, (15-12)^2$	9
Zero-loss w	$15 - 3w = 0$	$w = 5$
dL/dw	$-6(15 - 3w)$	at $w=4 \rightarrow$ -18
Update ($\eta=0.1$)	$4 - 0.1(-18)$	5.8

The negative gradient (-18) means the loss decreases as w increases, so the step moves w **toward 5** (the minimum). One step from 4 lands at 5.8.

Quick revision checklist

- $\sigma(0)=0.5$; " $\hat{p} \geq 0.5$ " $\Leftrightarrow$ " $z \geq 0$ ". More negative bias $\Rightarrow$ fewer activations.
- MSE = mean of squared errors; $\partial L / \partial \hat{y} = 2(\hat{y} - y)$.
- Feedforward: hidden units first; sigmoid gives nonlinearity + bounded (0,1) output.
- Gradient descent: $w \leftarrow w - \eta \cdot dL/dw$; gradient = local rate of change of loss.
- dL/dw for $L=(15-3w)^2$ is $-6(15-3w)$ — remember the chain-rule factor.